Clickpoint Chat Service & RAG AI Engine

Complete Developer Manual (A–Z): Vector Embeddings, Hybrid Retrieval, Session Store, Redis Caching, LLM Synthesis, & Auto-Reindexing

Service Module: clickpoint-chat-service (:8002)	**Embedding Model:** all-MiniLM-L6-v2 (384-dim)
Ingestion Module: clickpoint-ingestion-service (:8001)	**Vector Store:** PostgreSQL 16 + pgvector
LLM Provider: Google Gemini 1.5 Flash	**Response Cache:** Redis 7 + PostgreSQL

Executive Summary: *The Clickpoint Chat Service is a high-performance Retrieval-Augmented Generation (RAG) backend. It converts CMS data into 384-dimensional vector embeddings, executes hybrid candidate retrieval (pgvector + keyword domain boosting + SQL fallback), tracks multi-turn session history in PostgreSQL, caches frequent queries in Redis, and synthesizes grounded markdown responses using Google Gemini LLM.*

1. Microservices Architecture & Component Map

The system consists of two microservices working alongside a Next.js web application, PostgreSQL database, and Redis cache.

Service / Component	Tech Stack	Primary Responsibility
chat-service (:8002)	Python 3.9, FastAPI, Uvicorn, psycopg2	Handles visitor chat requests, session tracking, intent classification, vector retrieval, prompt building, LLM synthesis, and response caching.
ingestion-service (:8001)	Python 3.9, FastAPI, SentenceTransformers	Extracts data from 11 CMS PostgreSQL tables, performs single-entity chunking, generates 384d vector embeddings, updates pgvector, and flushes cache.
PostgreSQL + pgvector (:5433)	PostgreSQL 16, pgvector extension	Stores knowledge_chunks with 384d vectors, chat_sessions, and chat_messages history log.
Redis Cache (:6380)	Redis 7 Alpine	Provides sub-5ms exact string match response cache for frequent visitor questions.
Next.js Web App (:3001)	Next.js 14 App Router, React, TailwindCSS	Renders chatbot UI widget (chatbot-widget.tsx), handles session storage in localStorage, proxies chat API calls, and triggers auto-reindexing on admin updates.

2. Vector Embedding & Single-Entity Chunking Strategy

2.1 Embedding Model Specification

- **Model:** sentence-transformers/all-MiniLM-L6-v2
- **Dimensions:** 384 floating-point numbers.
- **Distance Metric:** Cosine Similarity (1 - (A · B) / (||A|| * ||B||)). In pgvector: (1 - (embedding <=> %s::vector)).
- **Performance:** ~12ms inference per batch, lightweight, low memory footprint.

2.2 Single-Entity Chunking Formula

Clickpoint uses a 1:1 granular single-entity strategy (1 team bio = 1 chunk; 1 job vacancy = 1 chunk; 1 service = 1 chunk). Text content is constructed using a rich structured template:

```
# Rich Embedding Text Construction (ingestion-service)
def build_embedding_text(chunk: dict) -> str:
    title = chunk.get("title") or ""
    source_type = chunk.get("source_type") or ""
    content = chunk.get("content") or ""
    # Formula: SOURCE_TYPE: Title\nContent
    return f"{source_type.upper()}: {title}\n{content}".strip()
```

3. Hybrid Retrieval Engine (Candidate Pool, Boosting, SQL Fallback)

The retrieval module (app/retrieval/retriever.py) implements a 3-stage hybrid search algorithm:

Step 1: Explicit Listing Intent Check

If query asks to 'list all services', 'show all jobs', or 'list all blogs', vector search is bypassed and a direct SQL query fetches all items for that source type ordered by title.

Step 2: Wide Candidate Fetching (pgvector)

Fetches a candidate pool of 40 nearest vector neighbors from knowledge_chunks without a restrictive similarity cutoff.

Step 3: Domain Keyword Boosting & Demotion Re-ranking

- **Leadership Queries:** Boosts profile/bio chunks containing leadership roles by +0.65. Demotes generic mission/stats chunks by -0.15.
- **Location Queries:** Boosts contact/office address chunks by +0.35.
- **Pricing Queries:** Boosts pricing/package chunks by +0.35.
- **Service Queries:** Boosts service chunks by +0.25.
- **SQL Fallback Candidate Injection:** If leadership keywords are present, explicitly injects top leadership rows from DB into the candidate pool if vector search ranked them low.
- **Top-K Selection:** Re-sorts candidates by boosted score and returns top 8 grounded chunks to LLM.

4. Session Management & Dialogue History Tracking

Multi-turn conversations are maintained seamlessly across user visits in app/storage/session_store.py.

- **Session ID:** Generated on frontend (or server-side UUIDv4 if missing). Preserved in client localStorage.
- **Message Storage:** Every user prompt and assistant reply is logged in PostgreSQL chat_messages table with timestamp and JSON metadata (matched source chunks).
- **Context Windowing:** On each new query, `get_session_history(session_id, limit=6)` retrieves the last 6 messages (3 turns) to pass to Gemini LLM for conversation context.

Table Name	Key Columns	Purpose
chat_sessions	id (UUID PK), createdAt (TIMESTAMP)	Tracks unique active user conversation threads.
chat_messages	id, sessionId (FK), role, content, matchedChunks (JSON), createdAt	Full turn-by-turn chat history log for audit and multi-turn LLM reasoning.
knowledge_chunks	id, title, content, sourceType, url, embedding (VECTOR 384)	Vector embedding store for semantic search.

5. Prompt Engineering & LLM Synthesis Pipeline

5.1 System Prompt Directives (Anti-Self-Introduction)

The system prompt in app/llm/prompts.py strictly enforces that answers must begin immediately with the core factual response without self-introductions ('Hello! I am Clickpoint AI...').

5.2 Multi-Layered Gemini LLM Execution Fallback

The provider module (app/llm/provider.py) executes a robust multi-level fallback chain:

1. **Gemini REST API Call:** Fast direct REST invocation over HTTPS with 35s timeout (`gemini-3.5-flash-lite`, `gemini-3.5-flash`, etc.).
2. **SDK Level 1 (`google.genai`):** Modern Google GenAI Python SDK.
3. **SDK Level 2 (`google.generativeai`):** Legacy GenerativeAI SDK fallback.
4. **Deterministic Short Fact Extractor:** If Gemini API is unreachable or key is missing, extracts bullet points directly from context chunks.

6. Dynamic CMS Auto-Reindexing & Cache Invalidation

When an admin modifies content anywhere in the Next.js CMS (Add/Edit/Delete job vacancy, blog post, service, team member, testimonial), the re-indexing pipeline updates the AI knowledge base automatically:

Step	Action	Technical Execution
1. CMS Event	Admin saves/deletes item in Next.js CMS	Server Action or API Route calls triggerReindexBackground() in src/lib/reindex-trigger.ts.
2. Async Dispatch	Background task dispatches HTTP call	Fires non-blocking curl / fetch request to http://localhost:8001/reindex.
3. Vector Ingestion	Ingestion service reads DB & re-chunks	Queries all 11 Postgres tables, generates 384d vectors using all-MiniLM-L6-v2, updates knowledge_chunks.
4. Cache Purge	Flushes Redis cache & DB response cache	Executes r.flushall() in Redis and truncates question_cache table so outdated answers are never served.

Summary: The Clickpoint Chat Service & RAG Engine represents a state-of-the-art production AI assistant. It provides sub-millisecond cached responses, zero-latency CMS data synchronization, hybrid vector search with domain keyword re-ranking, and grounded LLM answers formatted in markdown.